

APOSTILA COMPLETA DE SQL

Fundamentos, exemplos práticos e explicações

Do básico ao avançado — do SELECT às Window Functions

Sumário

1. Introdução ao SQL e aos Bancos de Dados Relacionais
2. Tipos de Dados
3. DDL — Definição de Dados (CREATE, ALTER, DROP)
4. DML — Manipulação de Dados (INSERT, UPDATE, DELETE)
5. DQL — Consultas com SELECT
6. Filtros com WHERE e Operadores
7. Ordenação, LIMIT e DISTINCT
8. Funções de Agregação e GROUP BY / HAVING
9. JOINS — Combinando Tabelas
10. Subconsultas e CTEs (WITH)
11. Funções de Janela (Window Functions)
12. Views, Índices e Constraints
13. Transações e Controle (TCL)
14. Boas Práticas e Erros Comuns
15. Exercícios Resolvidos

1. Introdução ao SQL

SQL (Structured Query Language) é a linguagem padrão para interagir com bancos de dados relacionais como PostgreSQL, MySQL, SQL Server, Oracle e SQLite. Com ela é possível criar estruturas (tabelas), inserir, atualizar, excluir e consultar dados.

Bancos de dados relacionais

Um banco de dados relacional organiza informações em tabelas (relações), formadas por linhas (registros) e colunas (atributos). As tabelas se relacionam entre si por meio de chaves — a chave primária (PRIMARY KEY) identifica unicamente cada linha, e a chave estrangeira (FOREIGN KEY) referencia registros de outra tabela.

Categorias de comandos SQL

- DDL — Data Definition Language: CREATE, ALTER, DROP, TRUNCATE.
- DML — Data Manipulation Language: INSERT, UPDATE, DELETE.
- DQL — Data Query Language: SELECT.
- DCL — Data Control Language: GRANT, REVOKE.
- TCL — Transaction Control Language: COMMIT, ROLLBACK, SAVEPOINT.

Modelo usado nesta apostila

Vamos usar um cenário de loja online com três tabelas: clientes, produtos e pedidos.

```
clientes(id, nome, email, cidade, criado_em)
produtos(id, nome, categoria, preco, estoque)
pedidos(id, cliente_id, produto_id, quantidade, data_pedido)
```

2. Tipos de Dados

Cada coluna precisa de um tipo de dado, que define quais valores ela aceita e quanto ocupa em disco. Os principais grupos:

Categoria	Tipos comuns	Uso
Numéricos inteiros	INT, BIGINT, SMALLINT	IDs, quantidades, contadores
Numéricos decimais	DECIMAL(p,s), NUMERIC, FLOAT	Valores monetários, medidas
Texto	CHAR(n), VARCHAR(n), TEXT	Nomes, descrições, e-mails
Data e hora	DATE, TIME, TIMESTAMP	Datas de nascimento, logs
Lógico	BOOLEAN	Verdadeiro/Falso, ativo/inativo
Binário	BLOB, BYTEA	Imagens, arquivos

Dica: para dinheiro use DECIMAL (ex.: DECIMAL(10,2)) — evita erros de arredondamento típicos de FLOAT.

3. DDL — Definição de Dados

CREATE TABLE

Cria uma nova tabela definindo colunas, tipos e restrições (constraints).

```
CREATE TABLE clientes (  
  id      SERIAL PRIMARY KEY,  
  nome    VARCHAR(100) NOT NULL,  
  email   VARCHAR(150) UNIQUE NOT NULL,  
  cidade  VARCHAR(80),  
  criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Explicação linha a linha:

- SERIAL PRIMARY KEY: gera IDs automáticos e identifica unicamente cada cliente.
- NOT NULL: obriga o preenchimento do campo.
- UNIQUE: garante que não haja e-mails repetidos.
- DEFAULT CURRENT_TIMESTAMP: preenche a data/hora atual quando não informada.

ALTER TABLE

Altera a estrutura de uma tabela já existente.

```
ALTER TABLE clientes ADD COLUMN telefone VARCHAR(20);  
ALTER TABLE clientes DROP COLUMN telefone;  
ALTER TABLE clientes RENAME COLUMN nome TO nome_completo;  
ALTER TABLE clientes ALTER COLUMN cidade SET NOT NULL;
```

DROP e TRUNCATE

DROP remove a tabela inteira (estrutura + dados). TRUNCATE apaga todos os dados, mantendo a estrutura.

```
DROP TABLE clientes;  
TRUNCATE TABLE pedidos;
```

Cuidado: DROP e TRUNCATE são irreversíveis fora de uma transação.

4. DML — Manipulação de Dados

INSERT

```
-- Inserindo um registro
INSERT INTO clientes (nome, email, cidade)
VALUES ('Ana Souza', 'ana@email.com', 'São Paulo');

-- Inserindo vários de uma vez
INSERT INTO produtos (nome, categoria, preco, estoque) VALUES
('Notebook', 'Informática', 3500.00, 10),
('Mouse', 'Informática', 80.50, 50),
('Cadeira', 'Móveis', 750.00, 8);
```

As colunas podem ser omitidas se você informar valores para todas na mesma ordem da tabela, mas listá-las é mais seguro e legível.

UPDATE

```
-- Atualiza o preço do produto com id = 2
UPDATE produtos
SET preco = 90.00
WHERE id = 2;

-- Aumenta em 10% o preço de todos os itens de Informática
UPDATE produtos
SET preco = preco * 1.10
WHERE categoria = 'Informática';
```

Regra de ouro: nunca execute UPDATE sem WHERE, senão todas as linhas serão alteradas.

DELETE

```
DELETE FROM pedidos WHERE id = 15;
DELETE FROM clientes WHERE criado_em < '2020-01-01';
```

Assim como no UPDATE, o DELETE sem WHERE apaga tudo. Teste antes com um SELECT usando o mesmo filtro.

5. DQL — Consultas com SELECT

O comando SELECT é o mais usado no dia a dia. Estrutura básica:

```
SELECT coluna1, coluna2  
FROM tabela  
WHERE condicao  
ORDER BY coluna  
LIMIT n;
```

Exemplos

```
-- Tudo da tabela  
SELECT * FROM produtos;  
  
-- Colunas específicas  
SELECT nome, preco FROM produtos;  
  
-- Renomeando (alias)  
SELECT nome AS produto, preco AS valor  
FROM produtos;  
  
-- Calculando na hora  
SELECT nome, preco, preco * 0.9 AS preco_com_desconto  
FROM produtos;
```

O alias (AS) melhora a legibilidade e é útil quando você aplica funções ou combina colunas.

6. Filtros com WHERE e Operadores

Operadores de comparação

Operador	Descrição	Exemplo
=	Igual	preco = 100
<> ou !=	Diferente	cidade <> 'Rio'
>, <, >=, <=	Maior / menor	preco >= 500
BETWEEN a AND b	Intervalo (inclusivo)	preco BETWEEN 100 AND 500
IN (...)	Pertence à lista	cidade IN ('SP','RJ')
LIKE	Padrão de texto	nome LIKE 'A%'
IS NULL	Valor nulo	telefone IS NULL

LIKE — curingas

- % substitui qualquer sequência de caracteres.
- _ substitui exatamente um caractere.

```
SELECT * FROM clientes WHERE nome LIKE 'A%';    -- começa com A
SELECT * FROM clientes WHERE email LIKE '%@gmail.com';
SELECT * FROM clientes WHERE nome LIKE '_ na%';  -- 2ª e 3ª letra "na"
```

Operadores lógicos

```
SELECT *
FROM produtos
WHERE (categoria = 'Informática' OR categoria = 'Escritório')
AND preco < 1000
AND estoque > 0;
```

Use parênteses para deixar clara a precedência entre AND e OR.

NULL

NULL não é zero nem string vazia — é a ausência de valor. Comparações com = ou <> sempre retornam desconhecido; use IS NULL ou IS NOT NULL.


```
SELECT * FROM clientes WHERE cidade IS NULL;  
SELECT * FROM clientes WHERE cidade IS NOT NULL;
```

7. Ordenação, LIMIT e DISTINCT

ORDER BY

```
SELECT nome, preco FROM produtos ORDER BY preco;      -- crescente (ASC)
SELECT nome, preco FROM produtos ORDER BY preco DESC; -- decrescente
SELECT * FROM clientes ORDER BY cidade ASC, nome ASC; -- múltiplas colunas
```

LIMIT / OFFSET

Restringe quantas linhas voltam. Útil para paginação e para conferir amostras.

```
SELECT * FROM produtos ORDER BY preco DESC LIMIT 5; -- top 5 mais caros
SELECT * FROM produtos ORDER BY id LIMIT 10 OFFSET 20; -- página 3 (10 por página)
```

SQL Server usa TOP em vez de LIMIT: `SELECT TOP 5 * FROM produtos ORDER BY preco DESC`.

DISTINCT

Remove linhas duplicadas do resultado.

```
SELECT DISTINCT cidade FROM clientes;
SELECT DISTINCT categoria, estoque > 0 AS tem_estoque FROM produtos;
```

8. Funções de Agregação e GROUP BY

Principais funções

Função	Descrição
COUNT(*)	Conta linhas (inclui NULL)
COUNT(coluna)	Conta valores não nulos
SUM(coluna)	Soma valores numéricos
AVG(coluna)	Média aritmética
MIN / MAX	Menor / maior valor

GROUP BY

Agrupar linhas que têm o mesmo valor em uma coluna e permite aplicar agregações por grupo.

```
-- Total de produtos por categoria
SELECT categoria, COUNT(*) AS qtd, AVG(preco) AS preco_medio
FROM produtos
GROUP BY categoria;

-- Total gasto por cliente
SELECT cliente_id, SUM(quantidade * p.preco) AS total_gasto
FROM pedidos ped
JOIN produtos p ON p.id = ped.produto_id
GROUP BY cliente_id;
```

HAVING

Filtra grupos após a agregação. Enquanto WHERE filtra linhas, HAVING filtra grupos.

```
SELECT categoria, COUNT(*) AS qtd
FROM produtos
GROUP BY categoria
HAVING COUNT(*) > 3;
```

Ordem lógica de execução de uma query: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.

9. JOINS — Combinando Tabelas

JOINS unem dados de duas ou mais tabelas com base em uma condição, geralmente entre chave primária e chave estrangeira.

Tipo	O que retorna
INNER JOIN	Somente linhas com correspondência nas duas tabelas
LEFT JOIN	Tudo da esquerda + correspondências da direita (NULL se não houver)
RIGHT JOIN	Tudo da direita + correspondências da esquerda
FULL JOIN	Tudo dos dois lados; NULL onde não houver correspondência
CROSS JOIN	Produto cartesiano (todas combinações)

INNER JOIN

```
SELECT c.nome, p.data_pedido, pr.nome AS produto
FROM clientes c
INNER JOIN pedidos p ON p.cliente_id = c.id
INNER JOIN produtos pr ON pr.id = p.produto_id;
```

LEFT JOIN

Mostra todos os clientes, mesmo os que não fizeram pedidos.

```
SELECT c.nome, p.id AS pedido_id
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id;

-- Clientes sem nenhum pedido:
SELECT c.nome
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
WHERE p.id IS NULL;
```

Boas práticas em JOINS

- Sempre use aliases (c, p, pr) para deixar a query legível.
- Prefixe as colunas com o alias da tabela para evitar ambiguidade.

- Garanta que existam índices nas colunas usadas no ON — isso acelera muito a consulta.

10. Subconsultas e CTEs

Subconsulta no WHERE

```
-- Clientes que fizeram pelo menos um pedido
SELECT nome
FROM clientes
WHERE id IN (SELECT DISTINCT cliente_id FROM pedidos);

-- Produtos com preço acima da média
SELECT nome, preco
FROM produtos
WHERE preco > (SELECT AVG(preco) FROM produtos);
```

Subconsulta no SELECT

```
SELECT c.nome,
       (SELECT COUNT(*) FROM pedidos p WHERE p.cliente_id = c.id) AS total_pedidos
FROM clientes c;
```

CTE (Common Table Expression) — WITH

Uma CTE é uma consulta temporária nomeada. Deixa queries complexas mais legíveis e permite reutilizar o resultado.

```
WITH gastos AS (
  SELECT p.cliente_id, SUM(p.quantidade * pr.preco) AS total
  FROM pedidos p
  JOIN produtos pr ON pr.id = p.produto_id
  GROUP BY p.cliente_id
)
SELECT c.nome, g.total
FROM clientes c
JOIN gastos g ON g.cliente_id = c.id
WHERE g.total > 1000
ORDER BY g.total DESC;
```

CTEs podem ser recursivas (WITH RECURSIVE) — úteis para hierarquias como organograma ou árvore de categorias.

11. Funções de Janela

Window functions fazem cálculos em um conjunto de linhas relacionadas à linha atual, sem colapsar o resultado como GROUP BY faz. A cláusula OVER() define a janela.

Ranking

```
SELECT nome, categoria, preco,  
       ROW_NUMBER() OVER (PARTITION BY categoria ORDER BY preco DESC) AS  
posicao,  
       RANK() OVER (PARTITION BY categoria ORDER BY preco DESC) AS  
rank_preco  
FROM produtos;
```

- PARTITION BY divide os dados em grupos (categoria).
- ORDER BY dentro do OVER define a ordem dentro do grupo.
- ROW_NUMBER gera 1,2,3... RANK deixa "buracos" em empates. DENSE_RANK não deixa.

Agregação como janela

```
SELECT cliente_id, data_pedido,  
       SUM(quantidade) OVER (PARTITION BY cliente_id ORDER BY data_pedido) AS  
acumulado  
FROM pedidos;
```

Aqui somamos as quantidades acumuladas de cada cliente ao longo do tempo — sem perder as linhas individuais.

LAG e LEAD

```
SELECT data_pedido, quantidade,  
       LAG(quantidade) OVER (ORDER BY data_pedido) AS anterior,  
       LEAD(quantidade) OVER (ORDER BY data_pedido) AS proximo  
FROM pedidos;
```

12. Views, Índices e Constraints

VIEW

Uma VIEW é uma consulta salva com nome, usada como se fosse uma tabela. Ótima para simplificar consultas frequentes e centralizar regras.

```
CREATE VIEW vw_pedidos_detalhados AS
SELECT p.id, c.nome AS cliente, pr.nome AS produto,
       p.quantidade, pr.preco, p.quantidade * pr.preco AS total
FROM pedidos p
JOIN clientes c ON c.id = p.cliente_id
JOIN produtos pr ON pr.id = p.produto_id;

SELECT * FROM vw_pedidos_detalhados WHERE total > 500;
```

Índices

Índices aceleram buscas — funcionam como o índice de um livro. Custam espaço em disco e tornam INSERT/UPDATE um pouco mais lentos.

```
CREATE INDEX idx_pedidos_cliente ON pedidos(cliente_id);
CREATE UNIQUE INDEX idx_clientes_email ON clientes(email);
```

- Crie índices em colunas usadas em WHERE, JOIN e ORDER BY.
- Evite índices em colunas com poucos valores distintos (ex.: booleanos).

Constraints

Constraint	Função
PRIMARY KEY	Identifica unicamente cada linha
FOREIGN KEY	Referencia uma linha de outra tabela
UNIQUE	Impede valores repetidos
NOT NULL	Exige valor preenchido
CHECK	Regra personalizada (ex.: preco > 0)
DEFAULT	Valor padrão quando não informado

```
CREATE TABLE pedidos (
  id SERIAL PRIMARY KEY,
```



```
cliente_id INT NOT NULL REFERENCES clientes(id),
produto_id INT NOT NULL REFERENCES produtos(id),
quantidade INT NOT NULL CHECK (quantidade > 0),
data_pedido DATE DEFAULT CURRENT_DATE
);
```

13. Transações (TCL)

Uma transação agrupa comandos que devem ser executados como uma unidade — todos ou nenhum. Segue as propriedades ACID: Atomicidade, Consistência, Isolamento e Durabilidade.

```
BEGIN;

UPDATE contas SET saldo = saldo - 100 WHERE id = 1;
UPDATE contas SET saldo = saldo + 100 WHERE id = 2;

-- Se algo deu errado:
-- ROLLBACK;

COMMIT;
```

SAVEPOINT permite pontos intermediários para rollback parcial.

```
BEGIN;
INSERT INTO pedidos (cliente_id, produto_id, quantidade) VALUES (1, 2, 5);
SAVEPOINT ponto1;

UPDATE produtos SET estoque = estoque - 5 WHERE id = 2;
-- ops, faltou algo
ROLLBACK TO ponto1;

COMMIT;
```

14. Boas Práticas e Erros Comuns

Boas práticas

- Escreva palavras-chave SQL em MAIÚSCULAS (SELECT, FROM) — facilita a leitura.
- Use nomes descritivos para tabelas e colunas (snake_case: data_pedido, cliente_id).
- Sempre teste UPDATE/DELETE antes com um SELECT usando o mesmo WHERE.
- Prefira JOIN explícito (JOIN ... ON) a JOIN implícito com vírgula.
- Nunca armazene senhas em texto puro — use hash (bcrypt, argon2).
- Comente queries complexas e formate em várias linhas.

Erros comuns

- Esquecer WHERE em UPDATE/DELETE — altera todas as linhas.
- Comparar NULL com = em vez de IS NULL.
- Confundir HAVING (grupos) com WHERE (linhas).
- Usar SELECT * em produção — sempre liste colunas necessárias.
- Ignorar índices em tabelas grandes: consultas ficam lentíssimas.

15. Exercícios Resolvidos

Base: as tabelas clientes, produtos e pedidos definidas nos capítulos anteriores.

Exercício 1 — Top 3 produtos mais caros

```
SELECT nome, preco
FROM produtos
ORDER BY preco DESC
LIMIT 3;
```

Ordenamos por preço decrescente e limitamos a 3 linhas.

Exercício 2 — Total de pedidos por cliente

```
SELECT c.nome, COUNT(p.id) AS total_pedidos
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
GROUP BY c.nome
ORDER BY total_pedidos DESC;
```

LEFT JOIN garante que clientes sem pedidos apareçam com 0.

Exercício 3 — Faturamento por categoria

```
SELECT pr.categoria, SUM(p.quantidade * pr.preco) AS faturamento
FROM pedidos p
JOIN produtos pr ON pr.id = p.produto_id
GROUP BY pr.categoria
ORDER BY faturamento DESC;
```

Exercício 4 — Clientes que gastaram mais que a média

```
WITH gastos AS (
  SELECT p.cliente_id, SUM(p.quantidade * pr.preco) AS total
  FROM pedidos p
  JOIN produtos pr ON pr.id = p.produto_id
  GROUP BY p.cliente_id
)
SELECT c.nome, g.total
FROM gastos g
JOIN clientes c ON c.id = g.cliente_id
WHERE g.total > (SELECT AVG(total) FROM gastos)
ORDER BY g.total DESC;
```

Exercício 5 — Produto mais vendido de cada categoria

```
WITH vendas AS (  
    SELECT pr.categoria, pr.nome,  
           SUM(p.quantidade) AS qtd,  
           ROW_NUMBER() OVER (  
               PARTITION BY pr.categoria  
               ORDER BY SUM(p.quantidade) DESC  
           ) AS rn  
    FROM pedidos p  
    JOIN produtos pr ON pr.id = p.produto_id  
    GROUP BY pr.categoria, pr.nome  
)  
SELECT categoria, nome, qtd  
FROM vendas  
WHERE rn = 1;
```

Combinamos GROUP BY (soma de quantidade) com uma window function para escolher o produto de maior venda por categoria.

Exercício 6 — Segundo maior preço (sem LIMIT)

```
SELECT MAX(preco) AS segundo_maior  
FROM produtos  
WHERE preco < (SELECT MAX(preco) FROM produtos);
```

Exercício 7 — Clientes sem pedidos

```
SELECT c.nome  
FROM clientes c  
LEFT JOIN pedidos p ON p.cliente_id = c.id  
WHERE p.id IS NULL;
```

16. SQL com Python

Este capítulo mostra como executar SQL a partir de Python usando três abordagens comuns: sqlite3 (built-in), psycopg2 (PostgreSQL) e SQLAlchemy (ORM e Core). Todos os exemplos são executáveis e seguem boas práticas de segurança (parâmetros, nunca concatenar strings).

16.1 sqlite3 — banco embutido, ideal para aprender

O módulo sqlite3 já vem com o Python. Não precisa instalar nada. Perfeito para praticar SQL localmente.

```
import sqlite3

# Conecta (cria o arquivo se não existir)
conn = sqlite3.connect("loja.db")
cur = conn.cursor()

# DDL - criando a tabela
cur.execute("""
CREATE TABLE IF NOT EXISTS clientes (
    id    INTEGER PRIMARY KEY AUTOINCREMENT,
    nome  TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    idade INTEGER
)
""")

# DML - inserindo com parametros (SEGURO contra SQL Injection)
cur.execute(
    "INSERT INTO clientes (nome, email, idade) VALUES (?, ?, ?)",
    ("Ana Silva", "ana@email.com", 29),
)
conn.commit()

# SELECT
cur.execute("SELECT id, nome, email FROM clientes WHERE idade >= ?", (18,))
for row in cur.fetchall():
    print(row)

conn.close()
```

Sempre use parâmetros — nunca concatene strings

ERRADO (vulnerável a SQL Injection):

```
email = input("email: ")
cur.execute("SELECT * FROM clientes WHERE email = '" + email + "'") # NUNCA
```

CERTO (o driver escapa os valores):

```
cur.execute("SELECT * FROM clientes WHERE email = ?", (email,))
```

16.2 Context manager e fetch*

Usar 'with' garante commit/rollback automático em sqlite3 e fechamento de recursos. Os métodos fetchone, fetchall e fetchmany controlam quanto trazer para memória.

```
import sqlite3

with sqlite3.connect("loja.db") as conn:
    conn.row_factory = sqlite3.Row # acessa colunas por nome
    cur = conn.cursor()

    cur.execute("SELECT id, nome, email FROM clientes ORDER BY nome")

    primeiro = cur.fetchone() # 1 linha
    print(primeiro["nome"], primeiro["email"])

    proximos = cur.fetchmany(10) # proximas 10
    todos = cur.fetchall() # o restante
```

16.3 executemany — inserção em lote

Para inserir muitos registros, executemany é ordens de magnitude mais rápido que chamar execute em loop.

```
dados = [
    ("Bruno Costa", "bruno@email.com", 34),
    ("Carla Dias", "carla@email.com", 27),
    ("Diego Ramos", "diego@email.com", 41),
]

with sqlite3.connect("loja.db") as conn:
    conn.executemany(
        "INSERT INTO clientes (nome, email, idade) VALUES (?, ?, ?)",
        dados,
    )
```

16.4 Transações — commit e rollback

Em uma transferência bancária, ou tudo é gravado, ou nada. Se algo falhar no meio, rollback desfaz as alterações.

```
import sqlite3

conn = sqlite3.connect("banco.db")
try:
    cur = conn.cursor()
    cur.execute("UPDATE contas SET saldo = saldo - 100 WHERE id = 1")
    cur.execute("UPDATE contas SET saldo = saldo + 100 WHERE id = 2")
    conn.commit()      # confirma
    print("Transferencia OK")
except Exception as e:
    conn.rollback()    # desfaz
    print("Falhou, revertido:", e)
finally:
    conn.close()
```

16.5 psycopg2 — conectando ao PostgreSQL

Para PostgreSQL em produção, o padrão é psycopg2 (ou psycopg 3). A API é parecida com sqlite3, mas usa %s como placeholder.

```
# pip install psycopg2-binary
import psycopg2
from psycopg2.extras import RealDictCursor

conn = psycopg2.connect(
    host="localhost",
    port=5432,
    dbname="loja",
    user="postgres",
    password="senha",
)

with conn, conn.cursor(cursor_factory=RealDictCursor) as cur:
    cur.execute(
        "SELECT id, nome, email FROM clientes WHERE idade >= %s",
        (18,),
    )
    for row in cur.fetchall():
        print(row["nome"], row["email"])

conn.close()
```


16.6 SQLAlchemy Core — SQL com portabilidade

SQLAlchemy permite escrever SQL de forma expressiva e trocar de banco (SQLite, Postgres, MySQL) sem mudar o código. O modo Core trabalha próximo do SQL.

```
# pip install sqlalchemy
from sqlalchemy import create_engine, text

engine = create_engine("sqlite:///loja.db", echo=False)

with engine.begin() as conn: # begin() faz commit no fim; rollback em erro
    conn.execute(text("""
        CREATE TABLE IF NOT EXISTS produtos (
            id    INTEGER PRIMARY KEY AUTOINCREMENT,
            nome  TEXT NOT NULL,
            preco REAL NOT NULL
        )
    """))
    conn.execute(
        text("INSERT INTO produtos (nome, preco) VALUES (:n, :p)"),
        [{"n": "Cafe", "p": 24.90},
        {"n": "Acucar", "p": 5.50},
        {"n": "Leite", "p": 6.20}],
    )

with engine.connect() as conn:
    result = conn.execute(text("SELECT nome, preco FROM produtos ORDER BY preco
DESC"))
    for nome, preco in result:
        print(f"{nome:10s} R$ {preco:.2f}")
```

16.7 SQLAlchemy ORM — objetos Python <-> tabelas

O ORM mapeia classes Python para tabelas. Você trabalha com objetos e o SQLAlchemy gera o SQL.

```
from sqlalchemy import create_engine, Integer, String, Float
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, Session

class Base(DeclarativeBase): pass

class Produto(Base):
    __tablename__ = "produtos"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    nome: Mapped[str] = mapped_column(String(100), nullable=False)
    preco: Mapped[float] = mapped_column(Float, nullable=False)
```

```

def __repr__(self):
    return f"<Produto {self.nome} R$ {self.preco:.2f}>"

engine = create_engine("sqlite:///loja.db")
Base.metadata.create_all(engine)

with Session(engine) as s:
    s.add_all([
        Produto(nome="Pao", preco=0.75),
        Produto(nome="Manteiga", preco=12.30),
    ])
    s.commit()

baratos = s.query(Produto).filter(Produto.preco < 10).all()
for p in baratos:
    print(p)

```

16.8 Pandas — SQL vira DataFrame

Para análise de dados, `pandas.read_sql` traz o resultado de uma query direto para um DataFrame.

```

# pip install pandas
import pandas as pd
from sqlalchemy import create_engine

engine = create_engine("sqlite:///loja.db")

df = pd.read_sql(
    "SELECT nome, preco FROM produtos WHERE preco > :p",
    engine,
    params={"p": 5.0},
)

print(df.head())
print("Preco medio:", df["preco"].mean())

# E o caminho inverso: DataFrame -> tabela
df.to_sql("produtos_backup", engine, if_exists="replace", index=False)

```

16.9 Boas práticas ao usar SQL em Python

- Sempre use parâmetros (?, %s, :nome) — nunca f-strings ou concatenação com dados do usuário.

- Use 'with' (context manager) para garantir commit/rollback e fechamento da conexão.
- Prefira executemany para cargas em lote — muito mais rápido que chamar execute em loop.
- Em produção, use pool de conexões (SQLAlchemy já faz por padrão).
- Guarde credenciais em variáveis de ambiente (os.environ) — nunca no código.
- Trate exceções específicas: sqlite3.IntegrityError, psycopg2.errors.UniqueViolation, etc.
- Para grandes resultados, itere linha a linha em vez de fetchall() para não estourar memória.

Fim da apostila — continue praticando com dados reais!